

# Học lập trình C

Lukas

2026-05-28T17:32:54Z

## Mục lục

|                                                                                            |           |
|--------------------------------------------------------------------------------------------|-----------|
| <b>Chương 1 - Giới thiệu và chương trình C đầu tiên</b>                                    | <b>2</b>  |
| <b>Chương 2 - Máy tính và lập trình cơ bản</b>                                             | <b>3</b>  |
| <b>Chương 3 - Lịch sử và triết lý của ngôn ngữ C</b>                                       | <b>5</b>  |
| <b>Chương 4 - Cú pháp &amp; cấu trúc chương trình C</b>                                    | <b>5</b>  |
| <b>Chương 5 - Điều khiển luồng (Control Flow)</b>                                          | <b>6</b>  |
| <b>Chương 6 - Hàm &amp; phạm vi biến (Functions &amp; Scope)</b>                           | <b>7</b>  |
| <b>Chương 7 - Mảng &amp; chuỗi (Arrays &amp; Strings)</b>                                  | <b>8</b>  |
| <b>Chương 8 - Con trỏ chi tiết (Pointers Deep Dive)</b>                                    | <b>9</b>  |
| <b>Chương 9 - Quản lý bộ nhớ và lỗi phổ biến</b>                                           | <b>10</b> |
| <b>Chương 10 - Struct, union, enum</b>                                                     | <b>11</b> |
| <b>Chương 11 - Header, Makefile, build systems</b>                                         | <b>12</b> |
| <b>Chương 12 - File I/O &amp; thao tác hệ thống</b>                                        | <b>12</b> |
| <b>Chương 13 - Xử lý lỗi &amp; ngoại lệ trong C</b>                                        | <b>13</b> |
| <b>Chương 14 - Tiền xử lý &amp; macro nâng cao</b>                                         | <b>14</b> |
| <b>và ##: stringification (#x) và token-pasting (a ## b) hữu ích cho code-gen trong C.</b> | <b>14</b> |
| <b>Chương 15 - Đa luồng &amp; đồng bộ (POSIX threads)</b>                                  | <b>14</b> |
| <b>Chương 16 - Mạng cơ bản (sockets)</b>                                                   | <b>15</b> |
| <b>Chương 17 - Tối ưu hóa &amp; profiling</b>                                              | <b>16</b> |
| <b>Chương 18 - Bảo mật &amp; an toàn bộ nhớ</b>                                            | <b>16</b> |
| <b>Chương 19 - Project: Trình biên dịch / Interpreter mini</b>                             | <b>17</b> |
| <b>Chương 20 - Project: Web server đơn giản</b>                                            | <b>18</b> |
| <b>Chương 21 - Testing &amp; CI cho C</b>                                                  | <b>18</b> |

## Chương 1 - Giới thiệu và chương trình C đầu tiên

Mục tiêu chương:

- Hiểu vai trò và lịch sử ngắn gọn của ngôn ngữ C.
- Thiết lập môi trường phát triển (Windows, Linux, macOS).
- Viết, biên dịch, chạy chương trình C đầu tiên.
- Giải thích chi tiết cấu trúc nguồn, hàm main, include, và cách dùng compiler flags cơ bản.

Tổng quan

Chương này hướng dẫn từ cơ bản nhất để một người mới - chưa biết lập trình - có thể: 1) chuẩn bị môi trường, 2) viết mã C đầu tiên, 3) biên dịch và chạy, và 4) đọc được các thông báo lỗi/cảnh báo cơ bản để sửa.

### 1. Vì sao học C?

- C là một ngôn ngữ hệ thống, gần với phần cứng, giúp hiểu sâu cách máy tính hoạt động (bộ nhớ, con trỏ, quản lý tài nguyên).
- Nhiều ngôn ngữ hiện đại dựa trên C (C++, C#, Java có nhiều ảnh hưởng), nên nắm C giúp dễ học các ngôn ngữ khác.

### 2. Công cụ cơ bản và chuẩn bị môi trường (chi tiết thực hiện)

- Cần: một trình soạn thảo (VSCode, Notepad++...), một terminal (PowerShell, bash), và một compiler (gcc/clang hoặc Visual Studio cl).

Windows (MSYS2 + MinGW-w64) - cách nhanh và tương thích: 1. Tải và cài MSYS2 từ <https://www.msys2.org> theo hướng dẫn. 2. Mở MSYS2 và cập nhật hệ thống: `pacman -Syu` (đóng/mở lại terminal nếu được yêu cầu). 3. Cài toolchain 64-bit: `pacman -S --needed mingw-w64-x86_64-toolchain mingw-w64-x86_64-make mingw-w64-x86_64-gdb`. 4. Mở “MSYS2 MinGW 64-bit” shell để làm việc với gcc 64-bit. 5. Kiểm tra: `gcc --version`, `make --version`, `gdb --version`

Windows (Visual Studio) - nếu muốn dùng `cl.exe`: - Cài Visual Studio Community với workload “Desktop development with C++”. Mở “Developer Command Prompt” để chạy `cl.exe`.

Linux (Debian/Ubuntu): - `sudo apt update && sudo apt install build-essential gdb valgrind make`

macOS (Homebrew): - `brew install gcc gdb make`

Lưu ý: Sau khi cài, hãy thêm đường dẫn compiler vào PATH nếu cần.

### 3. Cấu trúc chương trình C cơ bản - giải thích từng thành phần

Ví dụ tối thiểu (`hello.c`):

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Giải thích: - `#include <stdio.h>`: đưa vào các khai báo cho hàm nhập/xuất (`printf`, `scanf`). - `int main(void)`: hàm main trả về int; hệ điều hành sử dụng giá trị trả về để biết trạng thái chương trình (0 = thành công). - `printf`: in ra stdout; chuỗi có thể chứa escape sequences như `\n` (xuống dòng).

### 4. Biên dịch và các cờ thường dùng

- Cơ bản: `gcc -o hello hello.c`
- Với cảnh báo và chuẩn: `gcc -std=c11 -Wall -Wextra -O0 -g -o hello hello.c`
  - `-std=c11`: chọn chuẩn C
  - `-Wall -Wextra`: bật cảnh báo phổ thông

- -O0: tắt tối ưu để dễ debug
- -g: sinh symbol debug cho gdb

#### 5. Viết chương trình có input: ví dụ hello\_name.c

#include <stdio.h>

```
int main(void) {
    char name[128];
    printf("Nhập tên: ");
    if (fgets(name, sizeof(name), stdin) != NULL) {
        /* loại bỏ newline nếu có */
        size_t len = strlen(name);
        if (len > 0 && name[len-1] == '\n') name[len-1] = '\0';
        printf("Xin chào, %s!\n", name);
    }
    return 0;
}
```

Ghi chú an toàn: - Tránh dùng gets(); dùng fgets() để hạn chế buffer overflow. - Luôn kiểm tra giá trị trả về của hàm IO.

#### 6. Ví dụ và mã nguồn trong /code/chapter-01

- hello.c - “Hello, World!”
- hello\_name.c - đọc tên người dùng và in lời chào (dùng fgets)
- arithmetic.c - nhập hai số và in tổng/hiệu/tích/thương với kiểm tra chia cho 0
- input\_validation.c - ví dụ kiểm tra đầu vào hợp lệ

Mỗi ví dụ có README hướng dẫn biên dịch: gcc -std=c11 -Wall -Wextra -o exe file.c

#### 7. Bài tập thực hành (có lời giải trong /manuscript/solutions)

- Bài tập 1: Viết chương trình in “Hello, [Tên bạn]!” - dùng fgets để đọc tên.
- Bài tập 2: Viết chương trình nhập hai số nguyên, in tổng, hiệu, tích, thương (xử lý chia cho 0).
- Bài tập 3: Viết chương trình đọc một chuỗi và in độ dài (dùng strlen).

#### 8. Lỗi thường gặp và mẹo gỡ lỗi

- Dùng -Wall -Wextra để bắt sớm lỗi khả nghi.
- Nếu chương trình crash (segfault), chạy với gdb: gdb ./prog; run; backtrace
- Kiểm tra giá trị trả về của scanf/fgets để biết input có hợp lệ hay không.

Tài nguyên bổ sung

- K&R, “The C Programming Language”
- cppreference.com: trang tham khảo các hàm thư viện chuẩn

---

Ghi chú kỹ thuật: giữ ví dụ ngắn, có comment tiếng Việt; kiểm tra các ví dụ biên dịch trên MinGW và Linux.

## Chương 2 - Máy tính và lập trình cơ bản

Mục tiêu chương:

- Hiểu cách máy tính lưu trữ dữ liệu: bit, byte, biểu diễn số và ký tự.
- Nắm rõ bộ nhớ (stack/heap), kích thước kiểu dữ liệu và ý nghĩa của sizeof.
- Hiểu vòng đời chương trình: tiền xử lý -> biên dịch -> liên kết -> chạy.
- Thực hành: chuyển đổi số sang nhị phân, in bảng ASCII, và dùng malloc/free.

#### 1. Bit, byte và biểu diễn số

- Bit: đơn vị thông tin nhỏ nhất (0 hoặc 1). 1 byte = 8 bit.
- Endianness: little-endian (LSB lưu ở địa chỉ thấp) vs big-endian (MSB ở địa chỉ thấp). Thực hành: in các byte của một int để thấy thứ tự.
- Số nguyên dấu (signed) dùng two's complement để biểu diễn số âm; kích thước phụ thuộc vào nền tảng (sizeof(int)).

Ví dụ: kiểm tra sizeof trên nền tảng

```
#include <stdio.h>
int main(void) {
    printf("sizeof(char)=%zu, sizeof(int)=%zu, sizeof(long)=%zu\n", sizeof(char), sizeof(int), sizeof(long));
}
```

## 2. Ký tự và chuỗi

- C dùng char (thường 1 byte) để lưu ký tự; ASCII là tập ký tự cơ bản, UTF-8 là encoding phổ biến cho Unicode.
- Hàm tiện ích: isdigit(), isalpha(), isspace() (include <ctype.h>).
- Chuỗi trong C là mảng char kết thúc bởi '\0'.

## 3. Bộ nhớ: stack vs heap

- Stack: lưu biến cục bộ, tự động cấp phát/giải phóng khi rời scope.
- Heap: cấp phát động qua malloc/calloc/realloc; phải free thủ công.

Ví dụ phân bổ:

```
#include <stdlib.h>
#include <stdio.h>
int main(void) {
    int *arr = malloc(10 * sizeof(int));
    if (!arr) return 1;
    arr[0] = 42;
    free(arr);
    return 0;
}
```

## 4. Quá trình build: tiền xử lý -> compile -> link

- Preprocessing: mở rộng macro, include headers -> tạo file tạm (.i)
- Compiling: .i -> mã máy trung gian (.s) hoặc object (.o)
- Linking: nhiều .o + thư viện -> executable
- Công cụ: gcc -c file.c; gcc -o prog file.o -lm

## 5. Assembly và mối liên hệ với C (tóm tắt)

- Compiler sinh assembly; dùng gcc -S file.c để xem.
- Hiểu stack frame: lưu return address, local vars, và cách truyền tham số (platform dependent).

## 6. Bài thực hành

- bin\_convert.c: viết hàm chuyển số nguyên sang chuỗi nhị phân.
- ascii\_table.c: in bảng 0..127 với ký tự và mô tả.
- demo\_malloc.c: minh họa malloc/realloc/free và kiểm tra rò bộ nhớ (dùng valgrind trên Linux).

## 7. Lưu ý an toàn

- Tránh buffer overflow: luôn giới hạn kích thước khi đọc chuỗi.
- Sau malloc, kiểm tra con trỏ NULL.

Tài liệu tham khảo - K&R, cppreference.com, tài liệu gcc

---

Ghi chú: thêm minh họa bằng hình cho endianness và ví dụ gcc -S để sinh assembly.

## Chương 3 - Lịch sử và triết lý của ngôn ngữ C

Mục tiêu chương:

- Hiểu nguồn gốc, bối cảnh phát triển và các chuẩn chính của C.
- Nhận diện điểm khác biệt quan trọng giữa các chuẩn (K&R, C89/C90, C99, C11, C17) và ảnh hưởng tới code khi di chuyển.
- Biết cách chọn cờ biên dịch để tương thích và tránh undefined behavior.

### 1. Tóm tắt lịch sử

- 1970s: Dennis Ritchie phát triển C tại Bell Labs để viết hệ điều hành (Unix). C ra đời từ ngôn ngữ B và ảnh hưởng bởi ASSEMBLY nhưng có tính trừu tượng hơn.
- 1978: K&R (Kernighan & Ritchie) xuất bản cuốn “The C Programming Language” - tài liệu tham khảo ban đầu.
- 1989/1990: ANSI C (C89/C90) tiêu chuẩn hóa cú pháp và thư viện chuẩn, giúp code di động hơn.
- 1999: C99 thêm nhiều tính năng hiện đại (long long, // comment, restrict, inline, VLA).
- 2011: C11 thêm atomic và multi-thread hỗ trợ; C17 sửa lỗi và ổn định.

### 2. Triết lý thiết kế của C

- C theo đuổi hiệu năng và kiểm soát tài nguyên: cho phép thao tác gần phần cứng.
- Đơn giản về ngữ pháp nhưng mạnh mẽ về khả năng biểu diễn.
- Thiết kế theo kiểu “một số công cụ cho lập trình viên” thay vì che giấu mọi chi tiết.

### 3. Chuẩn C và tương thích

- Khi viết mã để chạy trên nhiều nền tảng, luôn xác định chuẩn biên dịch: gcc -std=c11 -Wall -Wextra
- Một số tính năng mới (VLA, long long) yêu cầu chuẩn tương ứng; nếu cần hỗ trợ compiler cũ, tránh hoặc viết fallback.
- Undefined behavior: hành vi không xác định (ví dụ: dereference NULL, buffer overflow) cần tránh - compiler có thể tạo ra các tối ưu khiến lỗi khó dò.

### 4. Ví dụ minh họa

- compatibility\_test.c: minh họa dùng long long và các macro kiểm tra kích thước kiểu.
- Dùng gcc -std=c99 compatibility\_test.c -o test để nhận biết khác biệt.

### 5. Bài tập và thí nghiệm

- Thử biên dịch cùng file với -std=c89, -std=c99, -std=c11 và ghi lại lỗi/cảnh báo.
- Dùng gcc -S để sinh assembly và đọc phần prologue/epilogue của hàm.

### 6. Lời khuyên cho lập trình viên mới

- Luôn đọc cảnh báo và sửa. Cảnh báo thường chỉ ra vấn đề tiềm ẩn.
- Tránh viết mã phụ thuộc vào kích thước kiểu (ví dụ assume int = 4 bytes). Dùng các kiểu cố định như int32\_t khi cần.

Tài nguyên

- K&R “The C Programming Language”
- cppreference.com và tài liệu GNU cho gcc

Ghi chú: thêm bảng so sánh tính năng giữa các chuẩn và timeline minh họa. - Lưu ý rõ các ví dụ lịch sử có thể không biên dịch trên compiler mặc định; ghi cách biên dịch (cờ -std) và mục đích giáo dục.

## Chương 4 - Cú pháp & cấu trúc chương trình C

Mục tiêu chương:

- Nắm rõ cú pháp cơ bản: tokens, khai báo, biểu thức, câu lệnh điều khiển.
- Hiểu tiền xử lý (#include, #define), include guards và cách tổ chức header/source.

- Thực hành tách module, viết prototype, và so sánh macro vs inline function.

#### 1. Tokens và cú pháp cơ bản

- Tokens gồm identifiers (tên biến/hàm), keywords (`int`, `return`, `if...`), literals (số, ký tự, chuỗi)
- Câu lệnh kết thúc bằng dấu chấm phẩy `;`. Block được bao bởi `{ ... }`.

#### 2. Kiểu dữ liệu và khai báo

- Kiểu cơ bản: `char`, `short`, `int`, `long`, `float`, `double`, `_Bool`, `void`.
- Phân biệt signed/unsigned, và dùng các kiểu kích thước cố định (`stdint.h`): `int8_t`, `int32_t`, `uint64_t`.
- Khai báo và khởi tạo biến: `int x = 0`; `const int MAX = 100`;

#### 3. Hàm, prototype và phạm vi

- Prototype: đặt ở header (.h) để các file khác biết về hàm.

Ví dụ: `int add(int a, int b);`

- Definition: `int add(int a, int b) { return a + b; }`

- Phạm vi (scope): local (biến trong hàm), file-scope (static biến ở đầu file), global (không nên dùng quá nhiều).

#### 4. Tiền xử lý

- `#include <stdio.h>` để lấy khai báo thư viện chuẩn.
- Include guard để tránh include lặp:

```
#ifndef MATH_UTILS_H #define MATH_UTILS_H /* declarations */ #endif
```

- `#define` cho macro; cẩn thận với side-effects.

Ví dụ macro nguy hiểm: `#define SQUARE(x) ((x)*(x))` // `SQUARE(i++)` sẽ có hành vi không mong muốn

#### 5. Macro vs inline function

- Macro: expand text, không có type-checking, có thể gây side-effect.
- Inline function (static inline): an toàn hơn, có type-checking và tối ưu hoá tương đương macro khi compiler cho phép.

Ví dụ: `static inline int square_int(int x) { return x * x; }`

#### 6. Tổ chức project nhỏ: header + source

- Thư mục: `src/`, `include/`, `tests/`
- Ví dụ: `math.h` (declarations), `math.c` (implementation), `main.c` (sử dụng)
- Makefile: rules để compile từng module và link thành executable.

#### 7. Bài tập

- Viết macro `SQUARE(x)` và minh hoạ lỗi khi dùng `SQUARE(i++)`.
- Viết inline function `square_int` và so sánh với macro.
- Tách chương trình thành `math.h/math.c/main.c` và build bằng Makefile.

Ghi chú: kèm theo ví dụ minh họa trong `/code/chapter-04`, mỗi ví dụ có README hướng dẫn biên dịch.

## Chương 5 - Điều khiển luồng (Control Flow)

Mục tiêu chương:

- Hiểu và phân biệt `if/else`, `switch`, `for`, `while`, `do-while`.
- Biết dùng `break` và `continue` hợp lý.
- Viết chương trình kiểm soát luồng với input hợp lệ và tránh infinite loop.

#### 1. If / else

Cấu trúc:

```
if (condition) { // code } else if (other_condition) { // code } else { // code }
```

- Condition là biểu thức trả về giá trị boolean (trong C, 0 = false, khác 0 = true).
- Short-circuit: với && và ||, C sẽ không đánh giá biểu thức phải nếu kết quả đã rõ.

Ví dụ: phân loại số

## 2. Switch

```
switch (expr) {  
    case 1: /* ... */ break;  
    case 2: /* ... */ break;  
  
default: /* ... */  
}
```

- Lưu ý fall-through: nếu thiếu break, execution sẽ tiếp tục case tiếp theo.
- Thường dùng cho lựa chọn rời rạc (menu, xử lý enum).

## 3. Vòng lặp: for, while, do-while

```
for (int i = 0; i < n; i++) { /* ... */ }  
while (condition) { /* ... */ }  
do { /* ... */ } while (condition);
```

- for thích hợp khi biết số lần lặp. while/do-while dùng cho lặp theo điều kiện.
- break: thoát vòng lặp; continue: bỏ phần còn lại và sang lần lặp tiếp theo.

## 4. Tránh infinite loop và lỗi logic

- Đảm bảo điều kiện vòng lặp sẽ thay đổi theo cách dẫn tới kết thúc.
- Thận trọng khi dùng biến unsigned trong phép so sánh (có thể vòng lặp không kết thúc).

## 5. Ví dụ thực hành

- In bảng cửu chương (for)
- Kiểm tra số nguyên tố (for with early exit)
- Menu đơn giản dùng switch

## 6. Bài tập

- 1) Viết chương trình in bảng cửu chương của 1..9 dùng for.
- 2) Viết hàm is\_prime(int n) và test từ 2..100.
- 3) Viết menu: [1] In bảng cửu chương, [2] Kiểm tra số nguyên tố, [0] Thoát.

Gợi ý biên dịch: gcc -std=c11 -Wall -Wextra -o prog file.c

Ghi chú: kèm file mẫu trong /code/chapter-05 có README hướng dẫn và test cases.

# Chương 6 - Hàm & phạm vi biến (Functions & Scope)

Mục tiêu chương:

- Hiểu cách thiết kế hàm, prototype, và cách tổ chức mã theo modules (.h/.c).
- Khác biệt giữa pass-by-value và việc truyền con trỏ để thay đổi giá trị bên ngoài.
- Hiểu storage class: auto, static, extern; và best practices khi dùng biến toàn cục.

## 1. Prototype và definition

- Prototype (khai báo) giúp compiler biết kiểu trả về và tham số trước khi gọi hàm.
- Đặt prototype trong header (.h), definition trong .c.

Ví dụ: `// math.h int add(int a, int b);`

`// math.c int add(int a, int b) { return a + b; }`

## 2. Truyền tham số: giá trị vs địa chỉ

- Mặc định: giá trị được truyền theo giá trị (copy). Thay đổi bên trong hàm không ảnh hưởng biến ở caller.
- Truyền con trỏ: hàm nhận địa chỉ để sửa giá trị ban đầu.

Ví dụ swap bằng con trỏ: `void swap(int a, int b) { int t = a; a = b; b = t; }`

## 3. Storage class và scope

- auto: mặc định cho biến cục bộ.
- static: nếu áp dụng cho biến file-scope, biến chỉ nhìn thấy trong translation unit; nếu cho biến cục bộ, biến giữ giá trị giữa các lần gọi.
- extern: khai báo biến toàn cục được định nghĩa ở file khác.

## 4. Function pointers

- Khai báo: `int (cmp)(const void, const void*)`;
- Dùng để truyền callback cho các hàm như qsort.

## 5. Tách module (header + source)

- Quy tắc: header cho API công khai; .c cho implementation nội bộ.
- Ví dụ cấu trúc: `include/math.h`, `src/math.c`, `examples/main.c`
- Makefile: compile `.c -> .o` và link thành exe.

## 6. Ví dụ thực hành

- Viết thư viện `vector_int`: create, push, pop, free; test memory leak.
- Viết swap bằng con trỏ và so sánh với macro swap.
- Dùng qsort với comparator function pointer.

## 7. Bài tập

- 1) Tạo module math với add/sub/mul/div (xử lý chia cho 0).
- 2) Viết `vector_int` (push/pop).
- 3) Viết comparator và sort array bằng qsort.

Ghi chú: đưa các ví dụ trong `/code/chapter-06` kèm Makefile và test cases.

# Chương 7 - Mảng & chuỗi (Arrays & Strings)

Mục tiêu chương:

- Hiểu cách khai báo, truyền và xử lý mảng tĩnh và động.
- Xử lý chuỗi (C-strings) an toàn, tránh buffer overflow.
- Hiểu pointer arithmetic và mối quan hệ giữa mảng và con trỏ.

## 1. Mảng tĩnh và khai báo

- Khai báo: `int a[10]; a[0]..a[9]`
- Lưu ý: truy cập ngoài ranh giới là undefined behavior.
- `size_t` và `sizeof` để xác định kích thước.

## 2. Mảng động (heap)

- Sử dụng `malloc/calloc/realloc` và `free`.
- Kiểm tra NULL sau malloc.
- Ví dụ: `int arr = malloc(n * sizeof *arr);`

## 3. Chuỗi (C-strings)

- Chuỗi là mảng char kết thúc bằng `'\0'`.
- Dùng `fgets` để đọc chuỗi an toàn: `fgets(buf, sizeof buf, stdin);`
- Hàm an toàn: `strncpy`, `strncat`, `snprintf`; nhưng hiểu giới hạn của từng hàm.

#### 4. Pointer arithmetic

- `p + i` tương đương `&p[i]`; `*(p + i) == p[i]`
- Dùng pointer khi cần hiệu năng hoặc thao tác mảng chung.

#### 5. Ví dụ thực hành

- `reverse_string.c`: đảo chuỗi nhập từ `stdin` bằng hoán vị cặp ký tự.
- `dynamic_string.c`: builder dùng `realloc` để nối chuỗi lớn.

#### 6. Bài tập

- Viết `safe_concat(dest, src, dest_size)` đảm bảo không tràn.
- Viết `dynamic vector int`: `push/pop/resize`.

Ghi chú: các ví dụ nằm trong `/code/chapter-07`; kèm README và test inputs.

## Chương 8 - Con trỏ chi tiết (Pointers Deep Dive)

### Mục tiêu

- Hiểu con trỏ: khai báo, dereference (`*`), address-of (`&`), pointer arithmetic.
- Biết con trỏ hàm, con trỏ tới mảng và mảng con trỏ.
- Nhận diện lỗi phổ biến: null pointer, uninitialized pointer, pointer to freed memory.

### Tổng quan

Con trỏ là tính năng mạnh mẽ của C cho phép thao tác trực tiếp với địa chỉ bộ nhớ. Sử dụng con trỏ giúp viết code hiệu năng cao và mô tả cấu trúc dữ liệu động (linked lists, trees), nhưng cũng dễ dẫn đến lỗi nghiêm trọng nếu xử lý không cẩn thận.

#### 1. Con trỏ cơ bản

- Khai báo: `int *p; char *s; void *pv;`
- Lấy địa chỉ: `int x = 5; int *p = &x;`

- Dereference: `*p` để truy cập/gán giá trị; luôn đảm bảo `p != NULL` trước khi deref.

Ví dụ:

```
int x = 10;
int *p = &x;
```

```
*p = 20; // x = 20
```

#### 2. Pointer arithmetic và mối quan hệ với mảng

- Nếu `p` là `int*`, thì `p + 1` tăng địa chỉ theo `sizeof(int)` bytes.
- Mảng khi truyền vào hàm sẽ hạ xuống thành pointer tới phần tử đầu; hàm nhận cả kích thước mảng để an toàn.

Ví dụ:

```
void print_array(int *a, size_t n) {
    for (size_t i = 0; i < n; ++i) printf("%d ", a[i]);
}
```

#### 3. Con trỏ hàm

- Khai báo: `int (cmp)(const void, const void*);`
- Dùng làm callback: `qsort`, `bsearch`, hoặc API event.

Ví dụ comparator cho qsort:

```
int cmp_int(const void *a, const void *b) {
    int ia = *(const int*)a;
    int ib = *(const int*)b;
    return (ia > ib) - (ia < ib);
}
```

#### 4. Lỗi phổ biến và cách phòng tránh

- Null dereference: luôn kiểm tra `p != NULL`.
- Use-after-free: sau khi `free(p)` gán `p = NULL`.
- Uninitialized pointer: khởi tạo pointer = NULL khi khai báo nếu chưa có địa chỉ.
- Buffer overflow: khi dùng pointer để viết vào vùng nhớ, đảm bảo chỉ ghi trong giới hạn.

#### 5. Ví dụ thực hành

- `pointer_examples.c`: minh họa truy cập, arithmetic, aliasing và cách tránh lỗi.
- `linked_list.c`: implement linked list với hàm `add/remove/print`; chú ý quản lý bộ nhớ.

#### 6. Công cụ debug và kiểm thử

- Valgrind (Linux) để dò rò bộ nhớ và use-after-free.
- ASAN (AddressSanitizer) khi build bằng clang/gcc với `-fsanitize=address`.

Ghi chú: kèm sơ đồ vùng nhớ (stack/heap/data/text) và ví dụ chạy được trong `/code/chapter-08`.

## Chương 9 - Quản lý bộ nhớ và lỗi phổ biến

Mục tiêu chương:

- Hiểu chính xác sự khác nhau giữa stack và heap, cách `malloc/calloc/realloc/free` hoạt động.
- Nhận diện và sửa các lỗi bộ nhớ phổ biến: memory leak, use-after-free, double-free, dangling pointer.
- Sử dụng công cụ dynamic (Valgrind, ASAN) và static (clang-tidy, cppcheck) để tìm lỗi.

#### 1. Stack vs Heap - minh họa

- Stack: lưu biến cục bộ, tự động giải phóng khi rời scope. Kích thước giới hạn.
- Heap: cấp phát động qua `malloc/calloc/realloc`; phải free thủ công.

Ví dụ ngắn:

```
int *make_array(size_t n) { // Trả pointer tới heap; caller phải free
    int *a = malloc(n * sizeof *a);
    if (!a) return NULL;
    return a;
}
```

#### 2. Các lỗi thường gặp (chi tiết & cách sửa)

- Memory leak: quên free. Cách: xác định ownership, mỗi `malloc` phải có free tương ứng, viết tests và dùng Valgrind.
- Use-after-free: dùng con trỏ sau khi free -> undefined behavior. Sửa: đặt pointer = NULL sau free; kiểm tra NULL.
- Double free: free cùng địa chỉ hai lần -> crash; giữ ownership rõ ràng.
- Dangling pointer: con trỏ trỏ tới vùng đã free; tránh bằng NULL và thiết kế API rõ ràng.

#### 3. Công cụ và cách dùng

- Valgrind (Linux): `valgrind --leak-check=full ./prog` - liệt kê leak, use-after-free.
- AddressSanitizer: `gcc -fsanitize=address -g -o prog prog.c` - phát hiện use-after-free, OOB.

- static analyzers: clang-tidy, cppcheck - tìm pattern nguy hiểm trước khi chạy.

#### 4. Patterns an toàn

- Kiểm tra kết quả malloc/calloc: `if (!ptr) { /* xử lý lỗi */ }`
- Dùng RAII-like patterns trong C: wrapper struct có hàm init/free để tránh leak.
- Dùng valgrind/asan trong CI cho bài tập và ví dụ.

#### 5. Ví dụ thực hành

- `/code/chapter-09/leak_example.c`: tạo leak cố tình và sửa.
- `/code/chapter-09/use_after_free.c`: minh họa và cách phát hiện bằng ASAN.
- `/code/chapter-09/tree.c`: cấp phát/giải phóng đúng cho binary tree.

#### Bài tập

- 1) Viết và test một binary tree: allocate và free mọi node; kiểm tra bằng Valgrind.
- 2) Viết ví dụ minh họa use-after-free và sửa nó bằng ownership rõ ràng.

Ghi chú: kèm README hướng dẫn sử dụng Valgrind và cách build với `-fsanitize=address`.

## Chương 10 - Struct, union, enum

#### Mục tiêu chương:

- Nắm vững cách khai báo và sử dụng struct, union, enum; hiểu alignment, padding và ảnh hưởng tới sizeof.
- Biết cách dùng struct để xây dựng cấu trúc dữ liệu (linked list, stack, queue) và quản lý memory cho chúng.

#### 1. Struct cơ bản

- Khai báo: `struct Point { int x; int y; };`
- Typedef để đơn giản: `typedef struct Point Point;`
- Truy cập: `p.x`, `p.y`; với pointer: `p->x`

#### 2. Padding & alignment

- Vì alignment, compiler có thể chèn padding giữa trường để thỏa điều kiện alignment.
- Dùng sizeof và offsetof để đo; sắp xếp trường (small->large) có thể giảm padding.

Ví dụ: trước/sau tối ưu layout

#### 3. Union và khi dùng

- union `U { int i; float f; char s[4]; }` - các trường chia sẻ cùng vùng nhớ.
- Dùng union cho variant hoặc overlay binary data; phải cẩn thận về field hiện tại hợp lệ.

#### 4. Enum và bitfield

- enum `Color { RED, GREEN, BLUE };`
- Bitfield: `struct Flags { unsigned a:1; unsigned b:1; };` chú ý portability giữa compiler.

#### 5. Ứng dụng: cấu trúc dữ liệu

- Linked list: `struct Node { int val; struct Node *next; };`
- Quy tắc: viết các hàm `create_node`, `insert`, `delete`, `free_list`; luôn free cả danh sách.

#### 6. Ví dụ thực hành

- `/code/chapter-10/linked_list.c`: insert/delete/find/iterate
- `/code/chapter-10/stack.c`: stack bằng dynamic array
- `/code/chapter-10/queue.c`: circular buffer implementation

#### Bài tập

- 1) Implement doubly linked list with splice.
- 2) Show size reduction by reordering fields in a struct and explain.

Ghi chú: kèm diagram memory và test cases.

## Chương 11 - Header, Makefile, build systems

Mục tiêu chương:

- Biết tổ chức dự án C nhiều file: header an toàn, tách interface/implementation.
- Viết Makefile cơ bản và giới thiệu CMake để build cross-platform.

### 1. Header design

- Include guard: `#ifndef MYLIB_H #define MYLIB_H ... #endif`
- Quy tắc include order: corresponding header first, then `<...>`, then project headers.
- Forward declaration để tránh include vòng.

### 2. Makefile cơ bản

- Variables: `CC = gcc, CFLAGS = -std=c11 -Wall -Wextra -g`
- Targets: `all, clean, test`
- Pattern rule: `%.o: %.c`
- Ví dụ Makefile cho project 3 module: `compile .c -> .o; link các .o`

### 3. CMake cơ bản

- CMakeLists.txt đơn giản: `project(name C); add_executable(); target_include_directories();`
- Lợi ích: tạo build dir, hỗ trợ nhiều generator, dễ tích hợp CI

### 4. Best practices

- Separate build directory (out-of-source build).
- Giữ header minimal; tránh include heavy headers trong header.
- Document API trong header comments.

Bài tập

- Viết Makefile cho project có 3 module (math, utils, main) và chạy build/test; tạo CMakeLists.txt tương đương.

Ghi chú: kèm sample Makefile và CMakeLists trong /code/chapter-11 để người học chạy.

## Chương 12 - File I/O & thao tác hệ thống

Mục tiêu chương:

- Đọc/ghi file text và binary bằng `fopen/fgets/fscanf/fprintf/fread/fwrite`.
- Hiểu buffering, modes, và xử lý lỗi I/O; khái niệm `mmap` và portability.

### 1. File text I/O

- `fopen(path, "r"/"w"/"a")` để mở file.
- `fgets` để đọc line an toàn; `fprintf` để ghi.
- Kiểm tra return của `fopen, fgets, fprintf`; dùng `perror` hoặc `strerror(errno)`.

### 2. Binary I/O

- `fread/fwrite` để đọc/ghi block bytes; hữu ích cho file nhị phân hoặc sao chép file.
- Ví dụ copy: đọc buffer 8KB và `fwrite` cho tới hết file.

### 3. Buffering và `fflush`

- `stdout` thường được buffered line-by-line hoặc fully buffered; `fflush` để đảm bảo ghi ra disk.

- File modes: “rb”/“wb” cho binary trên Windows

#### 4. mmap (khái niệm)

- mmap ánh xạ file vào bộ nhớ - nhanh cho đọc/ghi lớn; portability khác nhau giữa POSIX và Windows (MapViewOfFile).
- Giải thích ngắn, khuyến nghị dùng fread/fwrite cho portability trừ khi cần hiệu năng đặc biệt.

#### 5. Ví dụ thực hành

- /code/chapter-12/copy\_file.c: copy binary file with fread/fwrite.
- /code/chapter-12/csv\_parser.c: simple CSV reader handling quoted fields.

#### Bài tập

- Viết chương trình merge nhiều file text vào một file, xử lý lỗi I/O.
- Viết chương trình tìm kiếm chuỗi trong file lớn bằng buffer streaming.

Ghi chú: kèm README chỉ dẫn build/run và test files.

## Chương 13 - Xử lý lỗi & ngoại lệ trong C

#### Mục tiêu chương:

- Hiểu cách báo lỗi hiệu quả trong C: mã trả về, errno, perror, strerror và assert.
- Thiết kế API có kiểm tra lỗi rõ ràng và logging để dễ debug.

#### 1. Kiểm tra giá trị trả về và errno

- Luôn kiểm tra giá trị trả về của hàm I/O và hệ thống (fopen, malloc, read, write).
- errno (từ <errno.h>) chứa mã lỗi khi hàm hệ thống trả về lỗi; dùng perror hoặc strerror(errno) để in thông báo có nghĩa.

Ví dụ:

```
FILE *f = fopen(path, "r");
if (!f) { perror("fopen"); /* xử lý lỗi */ }
```

#### 2. assert vs runtime error handling

- assert() (từ <assert.h>) dùng để kiểm tra giả thiết nội bộ khi debug; không dùng assert cho kiểm soát luồng hoặc xử lý lỗi từ input.
- assert bị loại bỏ khi biên dịch với NDEBUG; dùng return codes và logging cho runtime.

#### 3. Thiết kế mã lỗi và API

- Trả về int/error enum hoặc struct { int code; char \*msg } cho các hàm phức tạp.
- Ví dụ:

```
typedef enum { ERR_OK = 0, ERR_IO = 1, ERR_OOM = 2 } error_t;
error_t read_config(const char *path);
```

- Document ownership: ai chịu free chuỗi lỗi, ai trả lỗi.

#### 4. Logging và debug macros

- Dùng macro LOG(level, fmt, ...) để chuẩn hoá thông báo. Ví dụ: #define LOG\_ERR(fmt, ...) fprintf(stderr, "ERROR:" fmt "\n", ##VA\_ARGS)
- Giữ logging nhẹ ở production hoặc điều khiển bằng macro #ifdef DEBUG.

#### 5. Best practices

- Kiểm tra mọi return từ hàm có thể fail.
- Trả mã lỗi rõ ràng, tránh dùng -1/0 nếu không có ý nghĩa.

- Thử nghiệm: viết test can lỗi (fault injection) để đảm bảo chương trình xử lý hỏng hóc.

Bài tập

- Viết wrapper `safe_fopen(path, mode, error_buf, bufsize)` - trả NULL on error và ghi thông báo người dùng vào `error_buf`.
- Viết chương trình demo: cố ý gây lỗi I/O (không tồn tại file) và hiển thị thông báo hữu ích.

Ghi chú: kèm ví dụ trong `/code/chapter-13` bao gồm `safe_fopen` và tests.

## Chương 14 - Tiền xử lý & macro nâng cao

Mục tiêu chương:

- Hiểu chi tiết về macro, các lỗi phổ biến, và khi nào nên dùng inline function thay vì macro.
- Học X-macros và conditional compilation để tổ chức code linh hoạt.

### 1. Parametric macros, stringification và token-pasting

- `#define SQR(x) ((x)*(x))` - ví dụ đơn giản nhưng có side-effects nếu x có tác dụng phụ.
- 

**và ##: stringification (#x) và token-pasting (a ## b) hữu ích cho code-gen trong C.**

### 2. Tránh side-effects trong macro

- Macro không kiểm tra kiểu và có thể evaluate tham số nhiều lần. Giải pháp: dùng inline function cho các phép toán an toàn.

### 3. X-macros pattern

- X-macros giúp giữ danh sách hằng số/structs/handlers ở một nơi và generate code lặp lại an toàn.
- Ví dụ: định nghĩa danh sách lỗi một nơi và generate enum + string table.

### 4. Conditional compilation

- `#ifdef FEATURE_X` để bật/tắt code theo nền tảng hoặc flags biên dịch.
- Sử dụng feature test macros để kiểm tra hàm thư viện của platform.

### 5. Ví dụ thực hành

- Viết `LOG(level, fmt, ...)` macro và so sánh với static inline void `log(level, const char*, ...)`.
- Dùng X-macro để generate enum và lookup table cho các lỗi.

Bài tập

- Viết X-macro cho danh sách lệnh CLI và generate switch-case xử lý tự động.

Ghi chú: kèm ví dụ trong `/code/chapter-14`, và so sánh compile-time outputs.

## Chương 15 - Đa luồng & đồng bộ (POSIX threads)

Mục tiêu chương:

- Hiểu cách tạo/hủy thread bằng pthreads, đồng bộ hóa bằng mutex và condition variables.
- Nhận diện race condition, deadlock và cách phòng tránh.

### 1. Tạo và quản lý thread

- `pthread_create`, `pthread_join`: tạo thread và đợi thread kết thúc.
- Truyền tham số vào thread bằng struct, tránh truyền địa chỉ biến cục bộ đã biến mất.

## 2. Mutex và bảo vệ shared data

- `pthread_mutex_t mutex`; `pthread_mutex_lock(&mutex)`; /\* critical section \*/ `pthread_mutex_unlock(&mutex)`;
- Giữ critical sections ngắn, tránh lock ordering khác nhau gây deadlock.

## 3. Condition variables

- `pthread_cond_wait`, `pthread_cond_signal`: dùng cho producer-consumer pattern.
- Luôn khóa mutex trước khi gọi `pthread_cond_wait`.

## 4. Race condition, deadlock và strategies

- Race: truy cập shared state không được bảo vệ.
- Deadlock: avoid by ordering locks consistently, using trylock or lock hierarchy.

## 5. Tools và debugging

- Helgrind (Valgrind tool) và ThreadSanitizer (TSAN) để phát hiện race.

### Bài tập

- Cài đặt producer-consumer với buffer giới hạn: multiple producers, multiple consumers, và kiểm thử với threads.

Ghi chú: kèm code trong /code/chapter-15 với Makefile để build trên POSIX; trên Windows hướng dẫn tương đương với Win32 threads.

## Chương 16 - Mạng cơ bản (sockets)

### Mục tiêu chương:

- Hiểu nguyên tắc hoạt động của TCP/UDP sockets và viết client-server đơn giản.
- Biết xử lý blocking vs non-blocking sockets và cơ chế select/poll.

### 1. Cơ bản về socket API

- `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `close()`.
- `sockaddr_in`, `htons/ntohs`, `inet_pton/inet_ntop` để chuyển đổi địa chỉ.

### 2. Giao tiếp: send/recv vs read/write

- send/recv cung cấp flags; read/write là wrapper POSIX.
- Luôn kiểm tra giá trị trả về - có thể partial send/recv.

### 3. Blocking vs non-blocking và multiplexing

- Blocking socket chặn khi không có dữ liệu; non-blocking trả -1 với EAGAIN/EWOULDBLOCK.
- `select()/poll()/epoll` để multiplex nhiều socket trong một thread.

### 4. Ví dụ thực hành

- `echo_server.c`: server TCP đơn giản multi-threaded (accept -> spawn worker thread)
- `http_request.c`: gửi HTTP GET tới server và in response headers

### Bài tập

- Viết server echo TCP xử lý nhiều client (thread-per-connection) và phiên bản dùng select để xử lý nhiều client trong một thread.

Ghi chú: kèm hướng dẫn build cho POSIX; trên Windows dùng Winsock (khác tên hàm: `WSAStartup`, `closesocket`).

## Chương 17 - Tối ưu hóa & profiling

Mục tiêu chương:

- Học cách đo hiệu năng thực tế, xác định hotspots và tối ưu hóa phù hợp.
- Hiểu khác biệt giữa tối ưu thuật toán và micro-optimizations (cache, inlining).

### 1. Đo và profiling

- Microbenchmark vs real workload: luôn test trên dữ liệu thực.
- Công cụ:
  - gprof: profiling cho chương trình được link với -pg (các hạn chế về inlining).
  - perf (Linux): đo CPU cycles, cache-misses, branch-misses.
  - Valgrind Callgrind + kcachegrind: profile gọi hàm và số lần gọi.

### 2. Algorithmic optimization

- Đổi thuật toán thường đem lợi lớn nhất ( $O(n^2) \rightarrow O(n \log n)$  ...).
- Trường hợp: dùng hash map thay vì tìm tuyến tính.

### 3. Micro-optimizations và cache-friendliness

- Data locality: tổ chức dữ liệu theo trình tự truy cập (AoS vs SoA).
- Prefetching, loop unrolling, inlining: cân nhắc trước khi áp dụng.
- Giữ critical loops đơn giản để compiler tối ưu.

### 4. Compiler optimizations and flags

- -O0, -O1, -O2, -O3, -Ofast, -march=native
- -flto (link-time optimization), -funroll-loops, -fprofile-guided
- Gợi ý: thử nhiều mức và profile để thấy tác động.

### 5. Benchmarking methodology

- Run multiple times, warm-up caches, pin CPU if cần.
- Use stable datasets and measure wall-clock and CPU time.

### 6. Ví dụ thực hành

- /code/chapter-17/sort\_bench.c: so sánh quicksort vs mergesort vs std qsort với dữ liệu lớn và đo thời gian.
- /code/chapter-17/cache\_demo.c: minh họa truy cập theo hàng vs cột trên matrix và đo cache effect.

Bài tập

- 1) Profile hàm tìm kiếm và thay bằng cấu trúc dữ liệu phù hợp; trình bày số liệu trước/sau.
- 2) Minh họa effect của memory layout bằng benchmark.

Ghi chú: kèm hướng dẫn chạy perf/gprof và cách đọc báo cáo.

## Chương 18 - Bảo mật & an toàn bộ nhớ

Mục tiêu chương:

- Nhận diện các lỗ hổng an toàn bộ nhớ phổ biến (buffer overflow, format string, integer overflow) và biện pháp phòng tránh.
- Sử dụng static và dynamic analysis để phát hiện lỗi (clang-tidy, cppcheck, ASAN).

### 1. Các loại lỗ hổng phổ biến

- Buffer overflow: viết quá giới hạn mảng/chuỗi  $\rightarrow$  remote code execution hoặc crash.
- Format string vulnerability: printf(user\_input)  $\rightarrow$  attacker kiểm soát format.
- Integer overflow/underflow: tính toán kích thước bộ nhớ dẫn tới allocation sai.

## 2. Defense-in-depth

- Biện pháp runtime: ASLR, DEP/NX, stack canaries, PIE.
- Biện pháp code: kiểm tra bound, dùng safe APIs (snprintf, strlen), validate input.

## 3. Static/dynamic analysis

- clang-tidy/cppcheck: phát hiện pattern xấu, buffer misuse.
- AddressSanitizer (ASAN): gcc/clang với -fsanitize=address để phát hiện OOB và use-after-free.

## 4. Secure coding guidelines

- Validate all inputs, sử dụng size\_t hợp lý, tránh signed/unsigned mismatch.
- Document ownership và lifetime của buffer/objects.
- Prefer bounds-checked APIs and explicit length tracking.

## 5. Ví dụ thực hành

- /code/chapter-18/buffer\_overflow\_demo.c: có lỗi có chủ đích và phiên bản đã vá.
- /code/chapter-18/format\_string\_demo.c: minh họa lỗi và fix bằng format-safe call.

### Bài tập

- Sửa một chương trình chứa buffer overflow; chứng minh bằng ASAN hoặc Valgrind.

Ghi chú: kèm hướng dẫn build với -fsanitize=address và cách đọc output ASAN.

## Chương 19 - Project: Trình biên dịch / Interpreter mini

### Mục tiêu chương:

- Xây một mini language step-by-step: lexer -> parser -> AST -> evaluator/codegen.
- Viết tests cho từng bước để đảm bảo đúng và có thể mở rộng.

### 1. Thiết kế ngôn ngữ con

- Bắt đầu với biểu thức số học, biến, và assignment: e.g.,  $x = 1 + 2 * (3 - 4)$
- Quy định grammar đơn giản (EBNF) và precedence.

### 2. Lexical analysis

- Viết lexer trả về token types: NUMBER, IDENT, PLUS, MINUS, STAR, SLASH, LPAREN, RPAREN, ASSIGN, SEMI.
- Xử lý whitespace và comments.

### 3. Parsing

- Dùng recursive-descent parser:  $\text{expr} \rightarrow \text{term} \{ (+|-) \text{term} \}$
- Xây AST nodes: Number, BinaryOp, Variable, Assign

### 4. Evaluation / Codegen

- Interpreter: traverse AST và evaluate; giữ environment (map var->value).
- Optionally: codegen to stack bytecode and implement VM loop.

### 5. Testing & incremental development

- TDD: viết tests cho lexer, parser and evaluator.
- Keep interfaces small and well-documented.

### 6. Ví dụ thực hành

- /code/chapter-19/lexer.c
- /code/chapter-19/parser.c
- /code/chapter-19/eval.c (main runs REPL)

### Bài tập dự án

- Bước 1: implement lexer and unit tests.
- Bước 2: parser + AST + evaluator; support variables.
- Bước 3: extend language: functions, if/else, while.

Ghi chú: kèm Makefile và test scripts in /code/chapter-19.

## Chương 20 - Project: Web server đơn giản

### Mục tiêu chương:

- Xây một web server nhỏ có thể serve static files và vài route động (JSON API).
- Hiểu trade-offs giữa thread-per-connection, thread-pool và non-blocking event loop.

#### 1. Architecture cơ bản

- Accept loop: accept() kết nối và xử lý request.
- Thread-per-connection: đơn giản nhưng tốn tài nguyên khi nhiều client.
- Thread pool: queue kết nối và worker threads xử lý; cân bằng throughput.

#### 2. HTTP request parsing

- Parsers đơn giản: đọc request line, headers, method, path.
- Không cần implement full HTTP/1.1 - focus vào GET và simple headers.

#### 3. Serving static files

- Map path -> filesystem path; validate path để tránh directory traversal.
- Set Content-Type theo file extension (MIME map).
- Use sendfile on POSIX for efficiency where có.

#### 4. Routing and logging

- Map routes to handler functions; basic router supports prefix/static routes.
- Logging: request time, method, path, response code.

#### 5. Ví dụ thực hành

- /code/chapter-20/simple\_server.c: thread-per-connection server serving files from ./www
- /code/chapter-20/thread\_pool\_server.c: server using fixed worker pool and request queue

### Bài tập

- Implement server serving static files and a /api/time JSON endpoint.
- Add graceful shutdown (signal handling) and basic rate-limiting.

Ghi chú: provide POSIX build instructions; Windows users see alternative using Winsock or run in WSL.

## Chương 21 - Testing & CI cho C

### Mục tiêu

- Thiết lập unit tests cho C và tích hợp CI (GitHub Actions).
- Viết testable code và mock dependencies nếu cần.

### Nội dung

1. Unit test frameworks: Unity, Check, Criterion
2. Viết test cases và test runner
3. GitHub Actions: workflow build, test, report
4. Coverage basics

Bài tập

- Thêm test cho một module và cấu hình GitHub Actions để chạy tests.

## **Chương 22 - Packaging & deploy**

Mục tiêu

- Đóng gói project C thành release: tarball, zip, hoặc installer.
- Hiểu cross-compilation và phát hành binary.

Nội dung

1. Tạo release: make dist, tar.gz, zip
2. Cross-compilation notes (mingw-w64, musl)
3. Building static binaries, distributing via GitHub Releases
4. Chuẩn bị metadata cho KDP (nội dung sách đã nêu ở phần đầu)

Bài tập

- Tạo một release script để build và tạo archive.